UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 09

Examen Parcial 01

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

¿Qué se evalúa?

Esta sesión es el primer examen parcial del curso. Los temas cubiertos son todo lo visto en las sesiones 01–08. El examen se resuelve **a mano, en papel**, sin computadora ni código.

Sesión(es)	Tema
01	La computadora y el modelo de memoria
02	C++ y los jueces en línea
03	Sistemas de numeración y conversiones
04	Representaciones binarias (complemento a 2, ASCII, IEEE 754)
05	Operadores de bits
06–08	Técnicas de bit manipulation

✔ Recuerda

El examen **no incluye escribir ni leer código fuente**. Es teórico y matemático: conversiones, operaciones a mano, razonamiento sobre patrones de bits, y preguntas conceptuales.

El modelo de la computadora

Componente	Analogía	Rol
CPU	Usuario sentado	Ejecuta instrucciones, una por una
RAM	Escritorio	Lo que está en uso *ahora*; se borra al apagar
Almacenamiento	Cajones	Permanente pero lento
Bus de datos	Manos del usuario	Transporta datos entre componentes
Sistema operativo	Reglas de la oficina	Administra el acceso a los recursos

- **Velocidades relativas**

  Si 1 instrucción de CPU tarda 1 segundo en escala humana:

Operación	Tiempo real	Escala humana
CPU (caché L1)	∼0.3 ns	1 segundo
Acceso a RAM	∼100 ns	∼6 minutos
Lectura en SSD	∼100 µs	∼4 días
Disco duro mecánico	∼10 ms	∼1 año
Red (mismo país)	∼50 ms	∼5 años

✔ Recuerda

En C++, **tú** decides qué entra y cuándo sale de la RAM (`new` y `delete`). En Python y Java un recolector de basura lo hace automáticamente. Ese control es la fuente de la velocidad de C++, y también de sus bugs más comunes.

Sistemas de numeración y conversiones

• Las cuatro bases del curso

Sistema	Base	Dígitos	Prefijo en C++
Binario	2	0, 1	`0b` (C++14)
Octal	8	0–7	`0` (cero inicial)
Decimal	10	0–9	(ninguno)
Hexadecimal	16	0–9, A=10, B=11 . . . F=15	`0x`

• Tabla de conversión rápida (hex ↔ binario)

Hex	Bin	Hex	Bin	Hex	Bin	Hex	Bin
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

• Potencias de 2 que debes memorizar

n	2^n	n	2^n	n	2^n
0	1	5	32	10	1 024
1	2	6	64	16	65 536
2	4	7	128	31	$\approx 2.1 \times 10^9$
3	8	8	256	32	$\approx 4.3 \times 10^9$
4	16	9	512	64	$\approx 1.8 \times 10^{19}$

• Métodos de conversión

Conversión	Método
Cualquier base → decimal	Suma los dígitos multiplicados por su peso posicional: $d_n \cdot b^n + \cdots + d_1 \cdot b^1 + d_0 \cdot b^0$
Decimal → base b	Divide repetidamente entre b; anota los restos. Lee de **abajo hacia arriba**.
Binario ↔ Octal	Agrupa en bloques de **3 bits** (de derecha a izquierda). Reemplaza directamente.
Binario ↔ Hex	Agrupa en bloques de **4 bits** (de derecha a izquierda). Reemplaza directamente.

$$6037_8 \xrightarrow{\text{3 bits}} 110\,000\,011\,111_2 \xrightarrow{\text{4 bits}} \underbrace{0001}_{1}\,\underbrace{1000}_{8}\,\underbrace{0011}_{3}\,\underbrace{1111}_{F} = \texttt{183F}_{16}$$

Para convertir entre octal y hexadecimal **siempre pasa por binario como puente**. Es directo y evita errores de aritmética.

Representaciones binarias

• Rango de valores con n bits

Tipo	Rango	8 bits	16 bits
Sin signo (*unsigned*)	0 a $2^n - 1$	0–255	0–65 535
Con signo (complemento a 2)	-2^{n-1} a $2^{n-1} - 1$	-128–$+127$	$-32\,768$–$+32\,767$

• Complemento a 2

Es la representación de negativos que usa **toda la computación moderna**. El bit más significativo es el bit de signo: 0 = positivo, 1 = negativo.

Para negar un número ($x \to -x$ y viceversa):

1. **Invierte** todos los bits.
2. **Suma 1** al resultado.

27_{10}	`0001 1011`
Invertir bits	`1110 0100`
Sumar 1	`1110 0101`

Verificación: $2^8 - 27 = 229 = \texttt{1110 0101}_2$. ✓

✔ Recuerda

En complemento a 2 con n bits:

- `1111...1` (todo unos) $= -1$
- `1000...0` (solo el bit de signo) $= -2^{n-1}$ (el más negativo)
- `0111...1` (signo 0, resto unos) $= 2^{n-1} - 1$ (el más positivo)
- El operador bitwise NOT satisface: `~x` $= -(x + 1)$ siempre.

Con 8 bits existe -128 pero **no** $+128$. El rango no es simétrico porque el cero ocupa un patrón del lado positivo. Intentar negar -128 produce -128 de nuevo (desbordamiento).

• ASCII — texto como números

Cada carácter es un byte. Los rangos clave:

Caracteres	Decimal	Hex
`'0'-'9'`	48–57	`0x30`–`0x39`
`'A'-'Z'`	65–90	`0x41`–`0x5A`
`'a'-'z'`	97–122	`0x61`–`0x7A`
`'\n'` (nueva línea)	10	`0x0A`
`' '` (espacio)	32	`0x20`

Mayúsculas $\leftrightarrow$ **minúsculas:** la única diferencia es el **bit 5** (peso $32 = 2^5$). Minúscula = Mayúscula $+ 32$.

Para pasar de un carácter dígito `'0'-'9'` a su valor entero: resta `'0'` (o aplica AND con `0x0F`): `'7' - '0' = 7`, `'7' & 0x0F = 7`.

• Fracciones en binario

Una fracción $\frac{1}{n}$ tiene representación **exacta** en base b solo si n no tiene factores primos distintos de los factores de b.

Fracción	¿Exacta en base 10?	¿Exacta en base 2?	Razón
$1/2$	Sí (0.5)	Sí (0.1)	Solo factor primo 2
$1/4$	Sí (0.25)	Sí (0.01)	Solo factor primo 2
$1/5$	Sí (0.2)	No ($0.\overline{0011}$)	Factor primo 5 ajeno a base 2
$1/3$	No ($0.\overline{3}$)	No ($0.\overline{01}$)	Factor primo 3
$1/10$	Sí (0.1)	No ($0.0\overline{0011}$)	Factor primo 5

Algoritmo: parte fraccionaria a binario (multiplica por 2):

1. Multiplica la parte fraccionaria por 2.
2. Escribe el dígito entero del resultado (0 ó 1).
3. Repite con la parte fraccionaria restante. Si llegas a 0, terminaste. Si el estado se repite, es periódico.

$$\begin{aligned} 0.625 \times 2 &= 1.25 \quad \rightarrow \text{bit } \mathbf{1}\text{, resto } 0.25 \\ 0.25 \times 2 &= 0.50 \quad \rightarrow \text{bit } \mathbf{0} \\ 0.50 \times 2 &= 1.00 \quad \rightarrow \text{bit } \mathbf{1}\text{, resto } 0 \end{aligned}$$

$0.625_{10} = 0.101_2$. Exacta (denominador $= 8 = 2^3$). ✓

• IEEE 754 — punto flotante

Tipo	Total	Signo	Exponente	Mantisa
`float`	32 bits	1	8 (bias 127)	23
`double`	64 bits	1	11 (bias 1023)	52

El número almacenado es $(-1)^s \times 1.mantisa_2 \times 2^{e-bias}$.

`0.1 + 0.2` $\neq$ `0.3` en casi cualquier lenguaje porque 0.1_{10} es periódico en binario. Al truncar la secuencia infinita en 23 (o 52) bits se introduce un error de redondeo. **Nunca compares flotantes con ==**; usa una tolerancia: $|a - b| < \varepsilon$.

Operadores de bits

• Tabla resumen

Nombre	Op.	Efecto aritmético	Descripción
Desp. izquierda	«	$x \times 2^k$	Mueve bits izq.; rellena con 0 a la derecha
Desp. derecha	»	$\lfloor x/2^k \rfloor$	Mueve bits der.; rellena con 0 a la izquierda
NOT bit a bit	~	$-(x+1)$	Invierte cada bit
AND bit a bit	&	—	1 solo si **ambos** bits son 1
OR bit a bit	|	—	1 si **al menos uno** es 1
XOR bit a bit	^	—	1 si los bits son **distintos**

• Tablas de verdad

a	b	a & b	a | b	a ^ b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

• Patrones especiales

- **Todo apagado:** `x = 0`
- **Todo encendido:** `x = -1` (en complemento a 2, $-1 = 2^n - 1 =$ todo 1s)
- **Solo el bit k:** `1 « k` $= 2^k$

• Precedencia (mayor a menor)

~ > «, » > & > ^ > |

Cuando combinas varios operadores en una expresión, usa **paréntesis**. La precedencia entre & y | es la fuente de errores más frecuente.

Técnicas de bit manipulation

• Las cuatro operaciones fundamentales sobre el bit k

Operación	Expresión	Por qué funciona
Leer bit k	`(x » k) & 1`	Desplaza el bit k al lugar 0; AND filtra el resto
Encender bit k	`x | (1 « k)`	OR con 1 siempre da 1; no toca los demás bits
Apagar bit k	`x & ~(1 « k)`	AND con 0 siempre da 0; ~ invierte la máscara
Invertir bit k	`x ^ (1 « k)`	XOR con 1 invierte; XOR con 0 deja igual

• Otras técnicas clave

Técnica	Expresión	Ejemplo
Máscara de k bits bajos	`(1 « k) - 1`	$k = 4$: `0000 1111` = 15
Apagar el bit encendido más bajo	`n & (n - 1)`	$n = 12$ (`1100`): result `1000`=8
Aislar el bit encendido más bajo	`n & -n`	$n = 12$ (`1100`): result `0100`=4
¿Es potencia de 2?	`n > 0 && !(n & (n-1))`	$n = 8$ ✓, $n = 7$ ×

• Propiedades del XOR

- `x ^ x = 0` (cualquier valor XOR consigo mismo da 0)
- `x ^ 0 = x` (XOR con cero deja el valor intacto)
- `(x ^ k) ^ k = x` (aplicar XOR dos veces con la misma clave regresa al original)

✔ Recuerda

XOR sirve para:

- **Invertir** bits seleccionados.
- **Intercambiar** dos variables sin variable temporal (`a ^= b; b ^= a; a ^= b`).
- **Encontrar el elemento único** en un arreglo donde todos los demás aparecen dos veces (XOR de todo el arreglo).
- Calcular la **distancia de Hamming** (a ^ b, luego contar los 1s del resultado).

• Las máscaras de posición par e impar

Constante	Bits encendidos
`0x55555555` = `0101 0101 ...`	Posiciones **pares** (0, 2, 4, . . .)
`0xAAAAAAAA` = `1010 1010 ...`	Posiciones **impares** (1, 3, 5, . . .)

n enchufes en serie forman un **contador binario**: después de c chasquidos, el enchufe i (contando desde 1) tiene el mismo estado que el bit $i-1$ de c. La lámpara en posición n está prendida si y solo si:

$$(\, c \gg (n-1)\,) \;\&\; 1 \;=\; 1$$

Ejercicios de repaso

Los siguientes ejercicios son del mismo tipo y dificultad que los del examen. Resuélvelos **a mano, en papel**.

• Conversión de bases

1. Convierte a binario: a) 173_{10} b) $\texttt{B4}_{16}$ c) 357_8

2. Convierte $0011\,1010\,1100\,0110_2$ a hexadecimal.

3. Convierte $\texttt{3F8A}_{16}$ a octal. *(Pista: pasa por binario.)*

4. En base 4, escribe el número $\texttt{FAD}_{16}$.

• Complemento a 2

5. Representa en **8 bits complemento a 2**: a) -43 b) -1 c) -128

6. ¿Qué valor decimal tiene `1111 0001` interpretado como entero con signo de 8 bits? Muestra cómo lo determines.

7. ¿Cuál es el rango de valores representables con 12 bits en complemento a 2?

• ASCII

8. Decodifica el mensaje: `50 52 4F 47 52 41 4D 41` (hex).

9. El byte `0x74` es una letra minúscula. Sin consultar la tabla, determina la mayúscula correspondiente y su valor hexadecimal. Explica el procedimiento con operaciones de bits.

10. ¿Cuántos bytes ocupa la cadena `"HOLA"` en memoria? ¿Es la misma cantidad que `"hola"`? Justifica.

• Fracciones e IEEE 754

11. Convierte a binario las siguientes fracciones; indica si el resultado es exacto o periódico y, si es periódico, cuál es el patrón:

a) $\frac{3}{4}$ b) $\frac{1}{6}$ c) $\frac{7}{8}$

12. Sin hacer la conversión, determina cuáles son exactas en binario. Justifica con los factores primos.

$$\frac{1}{32}, \quad \frac{1}{12}, \quad \frac{5}{16}, \quad \frac{1}{7}$$

13. Un `float` de 32 bits tiene el patrón:

$$\underbrace{1}_{signo}\ \underbrace{10000000}_{exp}\ \underbrace{10000000000000000000000}_{mantisa}$$

a) ¿Es positivo o negativo?

b) El exponente real es $e - 127$. ¿Cuánto vale aquí?

c) La mantisa representa $1.1_2 = 1.5_{10}$. ¿Qué número decimal es?

• Operadores y técnicas de bits

14. Calcula a mano. Muestra los 8 bits en cada paso.

a) `23 & 29`

b) `23 | 29`

c) `23 ^ 29`

d) `~ 23` (expresa en decimal)

e) `23 « 2`

f) `100 » 3`

15. Evalúa paso a paso: `(~(18 » 1)) & 55`

16. Dado $n = 44$ (`00101100`):

a) ¿Cuánto vale `n & (n-1)`? ¿Qué bit se apagó?

b) ¿Cuánto vale `n & -n`? ¿Qué representa?

c) ¿Es n una potencia de 2? Justifica usando la expresión anterior.

17. Explica con palabras y un ejemplo concreto por qué XOR de todos los elementos del arreglo {3, 7, 3, 9, 7} da el elemento único.

• Preguntas conceptuales

18. ¿Por qué las computadoras usan base 2 y no base 10? Explica en términos del hardware.

19. El estándar IEEE 754 utiliza un *bias* en el exponente. ¿Qué problema resuelve eso?

20. ¿Por qué el rango de complemento a 2 no es simétrico alrededor del cero? ¿Cuántos valores positivos hay y cuántos negativos?

21. ¿Qué efecto tiene `x ^ 0xFF` sobre los 8 bits bajos de `x`? ¿Y `x & 0xFF`?

Resumen exprés

✔ Recuerda

Conversiones:

- Cualquier base $\rightarrow$ decimal: suma pesos posicionales.
- Decimal $\rightarrow$ base b: divisiones sucesivas, leer restos de abajo arriba.
- Bin $\leftrightarrow$ Oct: grupos de 3 bits. Bin $\leftrightarrow$ Hex: grupos de 4 bits.

Complemento a 2 en n bits:

- Para negar: invertir + sumar 1.
- Rango: -2^{n-1} a $+2^{n-1} - 1$. `~x` $= -(x + 1)$.

ASCII clave: `'A'`=65, `'a'`=97, `'0'`=48. Mayús $\leftrightarrow$ minús: ± 32.
Exactas en binario: solo fracciones cuyo denominador es potencia de 2.
Operadores de bits:

- $\ll k$: $\times 2^k$. $\gg k$: $\lfloor \div 2^k \rfloor$.
- AND apaga, OR enciende, XOR invierte bits seleccionados.
- `x ^ x = 0`, `x ^ 0 = x`.

Técnicas:

- Leer bit k: `(x>>k)&1`. Encender: `x|(1<<k)`. Apagar: `x&~(1<<k)`. Invertir: `x^(1<<k)`.
- Potencia de 2: `n>0 && !(n&(n-1))`.
- Bit más bajo: `n&-n`. Apagar bit más bajo: `n&(n-1)`.